

98. Un fondu au noir, et retour

Quatre palettes à la suite, et l'écran s'éteint en douceur.

1. Ce qu'il faut comprendre

Un **fondu au noir** — entre deux niveaux, quand on perd — n'est qu'une suite de palettes, chacune un peu plus sombre que la précédente : chaque indice avance d'une nuance vers 3, jusqu'à ce que tout soit à 3.

Comme il n'y a que quatre nuances, le fondu n'a que **quatre pas**. On range les palettes dans des **tables gravées** (chapitre 4) : `N0[pas]` est la nuance de l'indice 0 au pas `pas`. Pas 0 : la palette d'origine ; pas 3 : tout noir.

Le fondu s'étale sur plusieurs images : c'est la **boucle qui dure plusieurs images** du chapitre 3. `sens` dit où l'on va, `attente` fait patienter 8 tours entre deux pas. A lance le fondu au noir, B le fait revenir.

`paletteFond` est appelé **à chaque image**, avec des valeurs lues dans les tables : on peut, puisqu'écrire la palette ne coûte presque rien. Le compilateur assemble l'octet à l'exécution quand les nuances ne sont pas connues d'avance.

Le **fondu au blanc** est le même avec des tables qui vont vers 0 : c'est l'effet de l'écran qui « flashe » quand une bombe explose.

À toi : écris les tables d'un fondu au blanc et branche-le sur SELECT.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <ecran>` → [27.1. La fonction `ecran\(\)` — éteindre et rallumer l'écran](#)
- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)
- `#include <bouton>` → [0.65.5. La fonction `bouton\(\)` — lire la manette](#)
- `#include <paletteFond>` → [60.8. La fonction `paletteFond\(\)` — les quatre nuances du fond](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **773 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	55 o	← la console, à l'allumage — la mise en route de la console
main()	244 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> — attend l'image suivante
AttendreVBlank	16 o	← <code>ecran()</code> — attend que l'écran soit libre
AttendreAcces	12 o	← <code>EcrireTexte</code> — attend de pouvoir écrire une case
AdresseCase	10 o	← <code>poser()</code> — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← <code>texte()</code> — écrit un texte, lettre par lettre
CopierTuiles	94 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← <code>bouton()</code> — lit les huit boutons

```

Tuiles                208 o ← CopierTuiles - 12 lettres sur 43, et 1 tuile dessinée
"A: FONDU AU NOIR"    16 o ← texte() - un texte gravé
"B: RETOUR"           9 o ← texte() - un texte gravé
N0_0                  4 o ← paletteFond() - une table gravée
N1_1                  4 o ← paletteFond() - une table gravée
N2_2                  4 o ← paletteFond() - une table gravée

```

Lettres gravées : ABDEFINORTU:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```

paletteFond() × 1    83 o sur place, dans main() + N0_0 (4 o), N1_1 (4 o), N2_2 (4 o)
texte()       × 2    22 o sur place, dans main() + EcrireTexte (10 o)
ecran()       × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
poser()       × 1    13 o sur place, dans main() + AdresseCase (10 o)
bouton()      × 2    10 o sur place, dans main() + LireManette (37 o)
image()       × 1     3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : A fait un fondu au noir (l'écran s'assombrit par étapes),
// B le fait revenir.
//
// L'idée : on ne redessine rien. On change seulement la palette, en 4 pas.
// Les trois tables N0, N1, N2 disent, pour chaque pas, quelle nuance montrer
// pour l'indice 0, l'indice 1 et l'indice 2 (l'indice 3 reste toujours 3, noir).

```

```

#include <Tuile>          // un dessin de 8 × 8 pixels
#include <ecran>          // éteint ou rallume l'écran
#include <texte>          // écrit un texte à l'écran
#include <poser>          // pose une tuile sur une case du fond
#include <bouton>         // lit un bouton de la manette
#include <paletteFond>    // choisit les quatre nuances du fond

```

2. UN DESSIN — BANDES

```

Tuile BANDES = {          // La bande des 4 indices, pour voir le fondu.
    "00000000", "00000000", "11111111", "11111111",
    "22222222", "22222222", "33333333", "33333333",
};

// Les quatre pas du fondu : chaque indice avance vers le noir.
//
//      pas :  0  1  2  3

```



à l'écran

3. UNE TABLE GRAVÉE DANS LA CARTOUCHE — N0

```

const uint8_t N0[] = { 0, 1, 2, 3 }; // l'indice 0 : 0, puis 1, puis 2, puis 3

```

4. UNE TABLE GRAVÉE DANS LA CARTOUCHE — N1

```

const uint8_t N1[] = { 1, 2, 3, 3 }; // l'indice 1 : 1, 2, 3, 3

```

5. UNE TABLE GRAVÉE DANS LA CARTOUCHE — N2

```

const uint8_t N2[] = { 2, 3, 3, 3 }; // l'indice 2 : 2, 3, 3, 3
// Au pas 3, tout vaut 3 : l'écran est noir.

```

6. UNE VARIABLE — PAS

```

uint8_t pas = 0; // 0 : normal ... 3 : tout noir

```

7. UNE VARIABLE — SENS

```
uint8_t sens = 0;    // 0 : on ne bouge pas ; 1 : on assombrit ; 2 : on éclaircit
```

8. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    ecran(0);                // Dessin écran éteint.
    texte(1, 1, "A: FONDU AU NOIR");
    texte(1, 2, "B: RETOUR");
    for (uint8_t c = 0; c < 20; c++) {
        poser(c, 7, BANDES);    // La bande, sur toute la ligne 7.
    }
    ecran(1);                // On rallume.

    uint8_t attente = 0;      // Pour ne changer de pas que toutes les 8 images.

    while (true) {            // La boucle du jeu :
        image();

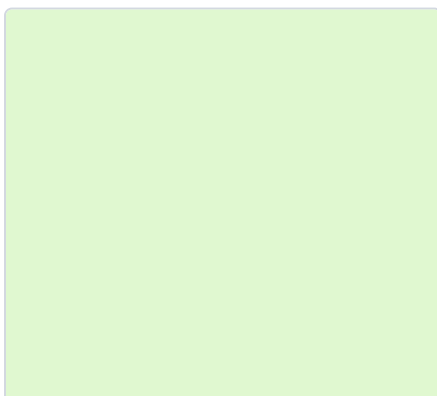
        if (bouton(A)) sens = 1;    // A : on va vers le noir.
        if (bouton(B)) sens = 2;    // B : on revient.

        attente++;
        if (attente >= 8) {        // Toutes les 8 images :
            attente = 0;
            if (sens == 1 && pas < 3) pas++; // un pas vers le noir (sans dépasser 3),
            if (sens == 2 && pas > 0) pas--; // ou un pas vers la lumière (sans passer sous 0).
        }

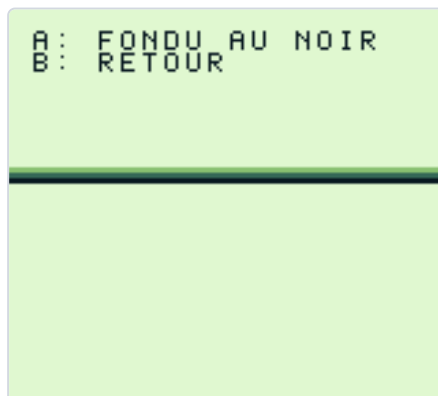
        paletteFond(N0[pas], N1[pas], N2[pas], 3); // On applique le pas actuel.
                                                    // pas = 1 → paletteFond(1, 2, 3, 3).
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



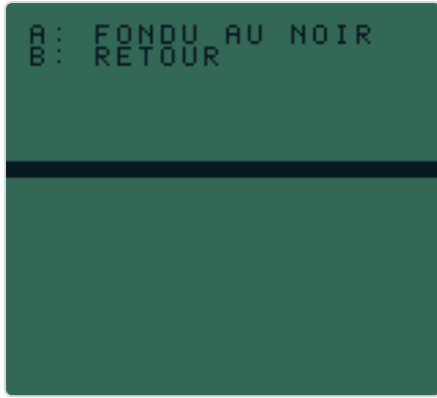
la toute première image — la console vient de démarrer — image 1



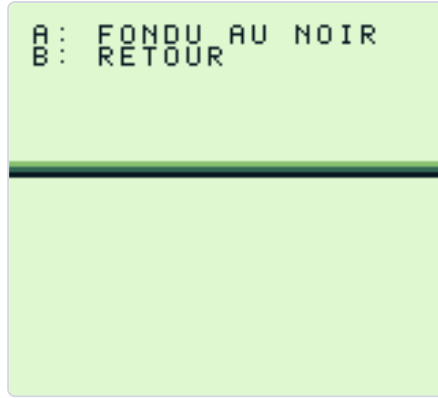
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé



B — l'écran a changé

5. Ce qu'on doit voir

A : l'écran s'assombrit en quatre pas jusqu'au noir complet. B : il revient.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, les quatre nuances
- ✓ en route, l'écran s'assombrit (2333)
- ✓ au bout, tout est noir
- ✓ B ramène la palette d'origine

Ensuite : **98.1. La fonction `paletteLutins()` — les nuances des lutins** — chapitre 15 : Les quatre nuances : la Game Boy d'origine. · [gameboy3](#) — écrire une cartouche Game Boy en C++